

Módulo 12 – Capítulo 06 – Sección 01

Containerización: Dockerfile del sistema, dependencias y configuración de producción

La containerización es el mecanismo que garantiza que el sistema que funciona en el entorno de desarrollo del ingeniero es exactamente el mismo sistema que se ejecuta en producción. "Funciona en mi máquina" es el síntoma de sistemas donde la containerización está incompleta o inconsistente — una dependencia que el desarrollador tiene instalada globalmente, una variable de entorno que existe en el shell del desarrollador pero no en el servidor, una versión de Python diferente entre local y producción. Un Dockerfile de producción correcto elimina esas fuentes de inconsistencia haciendo explícitas todas las dependencias del sistema.

El Dockerfile del proyecto usa un build multi-stage que separa el entorno de build del entorno de runtime. Esta separación tiene un beneficio de seguridad directo: el entorno de runtime no contiene las herramientas de compilación (gcc, build-essential, pip con todos sus archivos de caché) que son necesarias para instalar dependencias pero innecesarias para ejecutar la aplicación. Un contenedor más pequeño tiene menor superficie de ataque — menos binarios disponibles significa menos herramientas que un atacante puede usar si compromete el contenedor. El tamaño típico de la imagen final con multi-stage es de 200-300MB versus 800MB-1GB de un Dockerfile de stage único.

El **primer stage** (`builder`) usa `python:3.11-slim` como base, instala `build-essential` para compilar paquetes con extensiones nativas, y ejecuta `pip install --require-hashes -r requirements.txt`. La flag `--require-hashes` es crítica: requiere que cada paquete en `requirements.txt` tenga su hash SHA-256 especificado, verificado por pip durante la instalación. Si alguien modifica un paquete en el registry de PyPI después de la primera instalación — un ataque conocido como supply chain attack — la verificación de hash detectará la discrepancia y fallará la build. `requirements.txt` con hashes se genera con `pip-compile --generate-hashes` de `pip-tools`.

El **segundo stage** (`runtime`) usa `python:3.11-slim` limpio (sin `build-essential`), copia únicamente el directorio de `site-packages` del stage builder y el código de la aplicación. Configura un usuario no-root con `uid:gid 1000:1000` — un proceso que se ejecuta como root en un contenedor puede explotar vulnerabilidades de kernel para escapar al host; un proceso que se ejecuta como uid 1000 tiene capacidades limitadas incluso si logra explotar una vulnerabilidad del contenedor. El `ENTRYPOINT` configura uvicorn con 4 workers (adecuado para instancias `c5.xlarge` con 4 vCPU), sin `--access-log` (los logs de acceso se emiten via OpenTelemetry con mayor detalle y formato estructurado), y con `--timeout-keep-alive 30` para cerrar conexiones keep-alive inactivas que de otra forma consumen file descriptors.

Las variables de entorno sensibles nunca se incluyen en la imagen Docker — es uno de los errores de seguridad más frecuentes y con consecuencias más graves. Si `OPENAI_API_KEY` está en el Dockerfile, está en el layer history de la imagen, que es accesible para cualquiera que descargue la imagen. Las variables se inyectan en tiempo de ejecución mediante Kubernetes Secrets (sincronizados desde AWS Secrets Manager con External Secrets Operator en el Capítulo 6, Sección 5) o mediante Docker secrets en entornos de Docker Compose.

Aspectos técnicos de la containerización

- **Multi-stage build:** stage `builder` con `build-essential` + `pip install --require-hashes`; stage `runtime` con artefactos compilados únicamente — imagen final 200-300MB vs 800MB+ sin multi-stage.
- **Usuario no-root:** grupo y usuario `appuser` con `uid:gid 1000:1000`, sin acceso a `/root` ni a binarios de sistema — reduce el impacto de vulnerabilidades de contenedor.
- **Hash verification:** `pip install --require-hashes -r requirements.txt` — detecta tampering de paquetes en PyPI; `requirements.txt` generado con `pip-compile --generate-hashes` para reproducibilidad.
- **Health check integrado:** `HEALTHCHECK --interval=30s --timeout=5s --retries=3 CMD curl -f http://localhost:8000/health || exit 1` — Kubernetes usa este health check para determinar cuándo el pod está listo para recibir tráfico.
- **Configuración de producción:** `uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4 --timeout-keep-alive 30 --no-access-log` — workers configurados según vCPUs de la instancia, logs via OpenTelemetry.
- **Verificación pre-deploy:** el pipeline CI/CD ejecuta `docker inspect` para verificar que la imagen resultante tiene el usuario no-root configurado y que no hay secrets en las variables de entorno hardcodeadas.

Nota del Arquitecto: La verificación de hashes en las dependencias es el control de seguridad que más frecuentemente se omite porque parece excesivo para un proyecto interno. La realidad es que los ataques de supply chain a paquetes de PyPI ocurren con regularidad — el typosquatting (paquetes con nombres similares a los populares, como `requets` en lugar de `requests`) y la inyección de código malicioso en paquetes legítimos comprometidos son vectores reales documentados. Para un sistema que procesa documentación técnica interna de una organización y tiene acceso a credenciales de servicios externos, la integridad de las dependencias no es una preocupación académica. `pip-compile --generate-hashes` añade 2 minutos al proceso de generación del `requirements.txt` y 30 segundos a cada build — es la inversión de tiempo más barata por unidad de reducción de riesgo en el pipeline de build.

Un Dockerfile de producción no es el Dockerfile de desarrollo con un CMD diferente. Requiere multi-stage build, usuario no-root, hash verification de dependencias y configuración explícita del runtime para ser seguro, reproducible y eficiente. El pipeline CI/CD del Capítulo 6, Sección 2 construye esta imagen en el stage de build y la publica en el registry privado — el fundamento sobre el que todos los despliegues posteriores se construyen.

Para recordar: Un Dockerfile de producción no es el Dockerfile de desarrollo con CMD diferente — requiere multi-stage build, usuario no-root, hash verification de dependencias y configuración explícita del runtime para ser seguro y reproducible.

Módulo 12 – Capítulo 06 – Sección 02

CI/CD pipeline: build, test, evaluación y despliegue automatizado

El pipeline CI/CD es el mecanismo que convierte el código de un PR en un sistema desplegado en producción de forma controlada, verificada y auditable. Para sistemas de software convencionales, ese pipeline típicamente incluye build, test y deploy. Para sistemas de IA, el pipeline incluye un stage adicional que no existe en ningún sistema convencional: la evaluación automática de la calidad del modelo. Sin ese stage, el CI/CD puede desplegar una versión que introduce una regresión de faithfulness o de task completion que ningún test de software convencional detectaría — el sistema funciona correctamente desde el punto de vista del código pero produce respuestas de menor calidad.

El pipeline del proyecto usa GitHub Actions con cuatro stages secuenciales que deben pasar todos para que el deploy proceda. La secuencia es no arbitraria: falla rápido en las validaciones más baratas (linting tarda segundos) antes de las más costosas (evaluación RAGAS con 20 muestras tarda varios minutos).

El **stage de build** construye la imagen Docker con `docker buildx build --cache-from type=registry,ref={registry}/{image}:cache --cache-to type=registry,ref={registry}/{image}:cache` para aprovechar el caché de layers entre builds, reduciendo el tiempo de build de 4-5 minutos a 1-2 minutos en casos donde solo el código de la aplicación cambió. Después del build, Trivy ejecuta un scan de vulnerabilidades sobre la imagen: `trivy image --severity CRITICAL --exit-code 1` falla el pipeline si hay algún CVE de severidad crítica en los paquetes de la imagen. Un CVE crítico en una dependencia de Python puede ser explotable para comprometer el contenedor — el scan automático en cada build garantiza que las vulnerabilidades conocidas se detectan antes de llegar a producción. La imagen se publica en el registry privado con dos tags: el SHA del commit (inmutable, para reproducibilidad) y la rama (mutable, para referencias de deployment).

El **stage de test** ejecuta en secuencia: linting con `ruff check --select E,F,W,I` (errores, flakes, warnings y ordenación de imports), type checking con `mypy --strict` (garantiza que

los contratos de tipos Pydantic en las herramientas del agente son correctos), unit tests con pytest y cobertura mínima del 80% (`--cov-fail-under=80`), integration tests sobre el stack de test completo (Qdrant, Redis y PostgreSQL levantados con Docker Compose en el runner del CI), y un smoke test de la API que levanta el servidor con la imagen construida y verifica que `/health` responde con HTTP 200 en menos de 500ms.

El **stage de evaluate** es el diferenciador para sistemas de IA. Ejecuta el script de evaluación RAGAS sobre 20 muestras aleatorias del golden dataset usando la imagen del stage de build. El script llama al endpoint `/query` del sistema con cada una de las 20 queries, recoge las respuestas y calcula faithfulness y answer relevance con el framework RAGAS. Si faithfulness cae por debajo de 0.80 o answer relevance cae por debajo de 0.75, el stage falla y el deploy se bloquea. Los umbrales de 0.80 y 0.75 (ligeramente inferiores a los umbrales de producción de 0.85 y 0.80) se justifican estadísticamente por la varianza de muestreo en solo 20 pares — ver la explicación detallada en el Capítulo 3, Sección 5. Las métricas de evaluación se publican como artefactos del workflow de GitHub Actions y se envían a Slack mediante webhook: "Deploy {commit_sha}: faithfulness=0.87, answer_relevance=0.83 — PASSED".

El **stage de deploy** aplica los manifests de Kubernetes con `kubectl apply --server-side` (que usa Server-Side Apply para resolver conflictos de forma determinista) y espera la confirmación de que el rollout completó con `kubectl rollout status deployment/{name} --timeout=10m`. Si el rollout no completa en 10 minutos — porque los pods no pasan los health checks o porque hay un error en la imagen — el pipeline falla y el rollout se revierte automáticamente al estado anterior por el comportamiento default de Kubernetes deployments.

Etapas del pipeline CI/CD

- **Build:** `docker buildx build` con cache de layers, Trivy scan `--severity CRITICAL --exit-code 1`, push a registry con tags SHA y rama; duración aproximada 2-3 minutos con cache hit.
- **Test:** `ruff + mypy --strict + pytest --cov-fail-under=80` + integration tests (stack completo en Docker Compose) + smoke test API; duración aproximada 5-8 minutos.
- **Evaluate:** RAGAS evaluation sobre 20 muestras aleatorias del golden dataset con el sistema en producción; gate: `faithfulness < 0.80` o `answer_relevance < 0.75` bloquea el deploy; publicación de métricas como artefacto y notificación Slack.
- **Deploy:** `kubectl apply --server-side`, `kubectl rollout status --timeout=10m`; rollback automático si el rollout no completa; notificación Slack con resultado y enlace a

Grafana.

- **Notificaciones:** Slack webhook en cada stage con resultado, duración y enlace a los logs del workflow — el equipo puede ver el estado del deploy sin abrir la interfaz de GitHub.

Nota del Arquitecto: El stage de evaluate es el que más fricción introduce y el que más valor aporta. Los primeros deploy donde el gate bloquea un deploy porque faithfulness cayó de 0.87 a 0.79 van a generar frustración — "¿por qué el CI rechaza un cambio que no toca el pipeline RAG?". La respuesta es que un cambio en el system prompt del agente, una actualización de una dependencia, o un cambio en la configuración de la búsqueda híbrida puede afectar la calidad de las respuestas de formas que los tests de software no detectan. El gate de evaluación es exactamente el mecanismo que detecta esas degradaciones antes de que lleguen a los usuarios. Una vez que el equipo experimenta el primer bloqueo que previno una degradación real, la fricción inicial se convierte en confianza.

El pipeline CI/CD con gate de evaluación es el artefacto de MLOps que distingue un proyecto de IA maduro de uno que despliega por fe. El Capítulo 6 continúa con la infraestructura como código que provisiona el entorno donde este pipeline despliega — el cluster EKS, la base de datos PostgreSQL y la instancia de Qdrant Cloud que forman el entorno de producción.

Para recordar: El stage de evaluación automática en el pipeline CI/CD es lo que distingue el MLOps del DevOps para sistemas de IA — sin una puerta de calidad cuantitativa, los despliegues pueden degradar silenciosamente el comportamiento del modelo en producción.

Módulo 12 – Capítulo 06 – Sección 03

Infraestructura como código: Terraform para provisionar el entorno de producción

La infraestructura como código es el principio de que el entorno de producción — los servidores, redes, bases de datos y configuraciones que ejecutan el sistema — se define en archivos de configuración versionados que pueden auditarse, reproducirse y revertirse con el mismo rigor que el código de la aplicación. Un entorno de producción provisionado manualmente mediante clicks en la consola de AWS es un entorno que nadie puede reproducir exactamente — las configuraciones exactas de los security groups, los tamaños de instancia, los parámetros de RDS viven en la memoria de quien lo configuró y en la interfaz de AWS, no en el repositorio. Cuando ese entorno falla o cuando el equipo necesita provisionar un entorno de staging idéntico, la única opción es "intentar recordar cómo se configuró".

La infraestructura del sistema integrador se provisiona con Terraform usando el provider de AWS. El estado de Terraform se almacena en un S3 bucket con versionado habilitado (`versioning { enabled = true }`) y DynamoDB para locking distribuido. El versionado del S3 bucket permite recuperar el estado anterior si un `terraform apply` falla a mitad — sin versionado, un apply fallido puede dejar el estado corrupto sin forma de restaurarlo. El locking con DynamoDB previene que dos ingenieros ejecuten `terraform apply` simultáneamente sobre el mismo entorno — una condición de race que puede producir configuraciones inconsistentes.

Los recursos se organizan en tres módulos Terraform con responsabilidades claramente separadas. El **módulo networking** provisiona la VPC con subnets públicas (para el load balancer que recibe tráfico externo) y privadas (para los workers de Kubernetes y las bases de datos que no deben ser accesibles desde internet) en tres Availability Zones para alta disponibilidad. Los security groups definen el tráfico permitido entre capas: el load balancer acepta tráfico en el puerto 443 desde internet, los pods de EKS aceptan tráfico desde el load balancer, y RDS solo acepta tráfico desde los security groups de los pods.

El **módulo compute** provisiona el cluster EKS v1.30 con managed node groups de instancias `c5.xlarge` (4 vCPU, 8GB RAM), el cluster autoscaler configurado para escalar entre 2 y 10 nodos según la carga, y IRSA (IAM Roles for Service Accounts) para que los pods de Kubernetes puedan asumir roles IAM de AWS sin credenciales estáticas — el pod de la aplicación necesita acceso a AWS Secrets Manager para leer las API keys, y IRSA le otorga ese acceso mediante tokens temporales de IAM sin hardcodear credenciales.

El **módulo services** provisiona RDS PostgreSQL 16 Multi-AZ para el almacenamiento del estado del agente (LangGraph Persistence), los audit logs y la tabla de documentos procesados. La configuración Multi-AZ garantiza failover automático en menos de 60 segundos si la instancia primaria falla — sin Multi-AZ, un fallo de la instancia RDS haría que el agente perdiera su estado de sesión y que el sistema no pudiera registrar peticiones en el audit log. ElastiCache Redis se provisiona para las colas Celery del pipeline de ingesta. Qdrant Cloud se usa como servicio administrado para la base vectorial en producción — la responsabilidad de backups, actualizaciones y alta disponibilidad es de Qdrant Cloud, no del equipo.

La parametrización por entorno usa workspaces de Terraform: `terraform workspace select dev`, `staging` o `prod`, con archivos `.tfvars` por workspace que definen `instance_type`, `min_nodes`, `max_nodes`, `db_instance_class` y `multi_az`. En dev, los nodos son `t3.medium` (más baratos), sin Multi-AZ en RDS y sin cluster autoscaler; en staging son `c5.large` con los mismos parámetros de seguridad que producción pero sin Multi-AZ; en prod son `c5.xlarge` con Multi-AZ y autoscaler habilitados. Esta parametrización permite que dev sea barato para experimentación, staging sea fiel a producción para pruebas de integración, y prod esté configurado para alta disponibilidad real.

Componentes de infraestructura como código

- **State backend:** S3 bucket con versionado habilitado + DynamoDB table para state locking — prevención de race conditions en applies concurrentes y capacidad de restaurar estado anterior.
- **Módulo networking:** VPC con subnets públicas (load balancer) y privadas (EKS, RDS) en 3 AZs; NAT Gateway para acceso saliente de las subnets privadas; security groups con principio de mínimo privilegio por capa.
- **Módulo compute:** EKS v1.30 con managed node groups `c5.xlarge`, cluster autoscaler (min 2, max 10 nodos), IRSA para permisos IAM por pod sin credenciales estáticas.

- **Módulo services:** RDS PostgreSQL 16 Multi-AZ para estado del agente y audit logs; ElastiCache Redis para colas Celery; Qdrant Cloud como servicio administrado para la base vectorial.
- **Parametrización por entorno:** workspaces Terraform con `.tfvars` por entorno — dev barato (`t3.medium`, sin Multi-AZ), staging fiel a producción, prod con alta disponibilidad completa.

Nota del Arquitecto: La elección de Qdrant Cloud como servicio administrado para producción (en lugar de auto-hospedar Qdrant en EKS) merece justificación explícita porque tiene un costo directo. Qdrant Cloud cuesta aproximadamente 120 USD/mes para el tier necesario para este caso de uso; auto-hospedar Qdrant en EKS con EBS persistence costaría aproximadamente 60 USD/mes en recursos de compute y storage. La diferencia de 60 USD/mes compra: backups automáticos con punto de restauración, actualizaciones automáticas de versión, monitoreo de disponibilidad gestionado, y eliminación de la responsabilidad operativa de mantener un stateful set de Qdrant en Kubernetes — una de las operaciones más delicadas en infraestructura de Kubernetes porque involucra gestión de volúmenes persistentes. Para un equipo pequeño, ese trade-off es casi siempre favorable a favor del servicio administrado.

La infraestructura como código no es solo documentación ejecutable — es el control de versiones de la infraestructura. El `git log` del directorio `infra/terraform/` muestra exactamente qué cambió en el entorno de producción, cuándo y quién lo cambió, con la posibilidad de revertir cualquier cambio a través de `terraform apply` del commit anterior. El Capítulo 6 continúa con la estrategia de despliegue canary que minimiza el riesgo de cada cambio al sistema.

Para recordar: La infraestructura como código no es solo documentación ejecutable — es el control de versiones de la infraestructura, que permite auditar qué cambió, cuándo y por quién, y revertir a un estado anterior si un cambio causa un incidente.

Módulo 12 – Capítulo 06 – Sección 04

Estrategia de despliegue: canary para el sistema e ingeniería de costos como práctica operativa

La estrategia de despliegue no es solo una decisión técnica sobre cómo se actualiza el sistema — es la gestión del riesgo de cada cambio. Un despliegue que lleva inmediatamente el 100% del tráfico a la nueva versión maximiza la velocidad de distribución del cambio y también maximiza el impacto si la nueva versión tiene un defecto. Un despliegue canary que lleva el 5% del tráfico primero, observa las métricas durante 30 minutos y avanza solo si las métricas son aceptables, minimiza el impacto de un defecto a exponer como máximo el 5% de los usuarios durante el tiempo de observación.

El proyecto implementa una estrategia de despliegue canary con Argo Rollouts. La configuración define cuatro steps: 5% del tráfico durante 30 minutos de análisis, 25% durante 20 minutos, 50% durante 15 minutos, y 100% si todos los análisis pasan. El balanceo de tráfico entre la versión stable y la versión canary se implementa mediante `VirtualService` de Istio, que permite balanceo de tráfico a nivel de HTTP Request con la precisión necesaria para el 5% inicial. Argo Rollouts gestiona la progresión automática entre steps, ejecutando un `AnalysisTemplate` en cada step que consulta métricas de Prometheus para tomar la decisión de avanzar o revertir.

El `AnalysisTemplate` evalúa tres métricas en cada step de análisis. La primera es `error_rate`: la proporción de peticiones que responden con HTTP 5xx en la versión canary, medida como `rate(http_requests_total{status=~"5.."}[5m]) / rate(http_requests_total[5m])`; el umbral es 0.01 (1%). La segunda es `p95_latency`: el percentil 95 de la latencia de respuesta en la versión canary, medida con el histograma de Prometheus de `http_request_duration_seconds`; el umbral es 3500ms (500ms por encima del SLA de 3000ms para dar margen estadístico). La tercera es `faithfulness_ragas`: la métrica de calidad RAG calculada sobre queries muestreadas en tiempo real en la versión canary, comparada contra el baseline de la versión stable; el umbral es una caída máxima de 0.05 puntos respecto al baseline. Si cualquier métrica viola su umbral durante el análisis,

Argo Rollouts ejecuta un rollback automático: reencamina el 100% del tráfico a la versión stable en menos de 2 minutos sin intervención manual.

Para cambios en el pipeline RAG que requieren re-indexación (cambio de modelo de embedding o de parámetros de chunking), el canary no es suficiente — el cambio afecta la colección completa de Qdrant, no solo el código de la aplicación. Para esos cambios se usa una estrategia blue-green para la base vectorial: se provisiona una colección paralela en Qdrant (`knowledge_base_blue` o `knowledge_base_green`), se re-indexa el corpus completo con los nuevos parámetros, se valida la nueva colección con el golden dataset de 200 pares, y se hace un cutover atómico actualizando la variable de configuración `QDRANT_COLLECTION_NAME` en el ConfigMap de Kubernetes — todos los pods recargan la configuración sin reinicio y comienzan a usar la nueva colección. Si las métricas de calidad caen después del cutover, revertir es tan simple como actualizar el ConfigMap al nombre de la colección anterior.

Configuración de la estrategia canary

- **Argo Rollouts:** steps de 5% → 25% → 50% → 100% con pausa entre steps para análisis automático de métricas.
- **AnalysisTemplate:** consultas a Prometheus para `error_rate` (umbral < 1%), `p95_latency` (umbral < 3500ms) y `faithfulness_ragas` (umbral: caída < 0.05 vs baseline stable).
- **Rollback automático:** triggered cuando cualquier métrica del AnalysisTemplate viola su umbral; tiempo de rollback < 2 minutos sin intervención manual; notificación a Slack con causa del rollback.
- **Blue-green para cambios RAG:** colección paralela en Qdrant, re-indexación completa, validación con golden dataset, cutover atómico via ConfigMap — ver procedimiento detallado en el runbook del Capítulo 9.
- **Notificaciones:** Argo Rollouts hooks a Slack con step actual, métricas de análisis y estado del rollout en cada transición.

Ingeniería de costos: el constraint de 0.02 USD/petición como práctica operativa

El constraint de costo de 0.02 USD por petición definido en el Capítulo 1 no es una aspiración — es un requisito que debe monitorearse y gestionarse activamente en producción. El costo real por petición es la suma de cuatro componentes: el costo de embedding de la query con

text-embedding-3-small (0.02 USD por millón de tokens $\times$ tokens de la query $\approx$ 0.00001 USD por petición típica), el costo de reranking con Cohere sobre top-20 candidatos (aproximadamente 0.001 USD por 1000 búsquedas $\approx$ 0.000001 USD por petición), el costo de generación con GPT-4o (5 USD por millón de tokens de input $\times$ tokens de contexto + prompt $\approx$ 0.012 USD por petición con contexto de 2400 tokens, más 15 USD por millón de tokens de output $\times$ tokens de respuesta $\approx$ 0.004 USD por respuesta de 250 tokens), y el costo de infraestructura prorrateado por petición (instancias EKS + RDS + Redis + Qdrant Cloud $\approx$ 0.002 USD/petición a 5000 peticiones/día). La suma típica es entre 0.015 y 0.020 USD por petición, con el LLM de generación aportando el 80% del costo.

El monitoreo de costos en producción usa AWS Cost Explorer para el costo de los servicios de AWS (EKS, RDS, ElastiCache) y métricas custom de Prometheus para el costo de las APIs externas. Para las APIs de OpenAI y Cohere, el sistema registra los tokens consumidos por petición (disponibles en los responses de la API) y los convierte a USD usando las tarifas vigentes almacenadas como ConfigMap — cuando las tarifas cambian, solo se actualiza el ConfigMap. El dashboard de Grafana incluye un panel de "Costo estimado por hora" calculado como `sum(tokens_used_by_model * price_per_token) / time_window`, con una línea de referencia en el costo esperado y una alerta cuando el costo por hora supera 1.5 veces el promedio de las últimas 24 horas.

La atribución de costos por usuario o por equipo es posible porque el JWT incluye el claim `team` de cada usuario. El pipeline de métricas registra los tokens consumidos por `team` y calcula el costo por equipo semanalmente. Este breakdown permite que el equipo de infraestructura facture internamente a los equipos que usan el sistema de forma intensiva (chargeback interno) y detectar usuarios o equipos con patrones de uso anómalos — un equipo que consume 10x el costo promedio puede indicar un caso de uso mal configurado o un abuso del sistema.

Nota del Arquitecto: El costo de LLM en producción tiene una propiedad que el costo de infraestructura convencional no tiene: puede crecer de forma discontinua y difícil de predecir cuando los usuarios formulan queries complejas que generan contextos extensos o que llevan al agente a ejecutar múltiples iteraciones. Un sistema con 50 usuarios concurrentes haciendo queries simples (1 iteración, 2000 tokens de contexto) tiene un costo muy diferente al mismo sistema con 50 usuarios concurrentes haciendo queries complejas (5 iteraciones, 6000 tokens de contexto cada una). El monitoreo de "costo por hora $>$ 1.5x del promedio" es la alerta que detecta este tipo de shift en el perfil de uso antes de que se refleje en la factura

mensual. AWS Budgets configurado con una alerta de gasto mensual proyectado es la segunda línea de defensa para prevenir sorpresas en la factura.

La ingeniería de costos es una práctica operativa continua, no una verificación puntual. El constraint de 0.02 USD/petición debe monitorearse en el dashboard, alertarse cuando se aproxima o supera, y gestionarse con las mismas herramientas de ingeniería que cualquier otro SLA del sistema. El Capítulo 8 profundiza en el monitoreo de costos como dimensión adicional del dashboard de observabilidad.

Para recordar: El canary deployment con análisis automático de métricas detecta degradaciones de calidad antes de que afecten al 100% de los usuarios, con rollback automático que no requiere intervención manual — el control más efectivo de riesgo de despliegue para sistemas de IA.

Módulo 12 – Capítulo 06 – Sección 05

Gestión de configuraciones: variables de entorno, secrets management y configuración por entorno

La gestión de configuraciones es el área donde los sistemas de producción fallan de formas que el desarrollo nunca anticipa. Las tres fuentes más comunes de incidentes de configuración en producción son: un secret hardcodeado en el código que se expone en el repositorio, una variable de entorno que varía entre dev y prod sin documentación explícita de las diferencias, y un cambio de configuración en producción que no tiene audit trail — alguien cambió el `OPENAI_API_KEY` en la consola de AWS y nadie recuerda cuándo ni por qué. El sistema integrador previene las tres con prácticas específicas.

El principio base es el de los Twelve-Factor App: toda la configuración que varía entre entornos se almacena en variables de entorno, nunca en el código ni en archivos commiteados al repositorio. Para el sistema integrador, la configuración tiene dos categorías: **secrets** (información sensible que nunca debe aparecer en texto plano en ningún artefacto versionado) y **parámetros no sensibles** (valores que configuran el comportamiento del sistema pero no son credenciales). La distinción determina el mecanismo de gestión.

Los secrets (`OPENAI_API_KEY` , `QDRANT_API_KEY` , `DATABASE_URL` con usuario y contraseña, `JWT_PRIVATE_KEY`) se almacenan en AWS Secrets Manager. El acceso desde los pods de Kubernetes se implementa con External Secrets Operator (ESO): un CustomResourceDefinition `ExternalSecret` define el mapping entre el nombre del secret en AWS SM y el Kubernetes Secret resultante. ESO sincroniza automáticamente el Kubernetes Secret cuando el secret en AWS SM cambia, sin necesidad de reiniciar los pods — el pod lee la variable de entorno del Secret montado como volume, y cuando el Secret se actualiza, Kubernetes propaga la actualización a los pods en ejecución. Este mecanismo de rotación sin downtime es crítico para la operación continua: rotar la API key de OpenAI cada 90 días no debe requerir un deployment del sistema.

La rotación automática de secrets se configura en AWS Secrets Manager con una Lambda function que genera nuevas API keys y actualiza el secret en AWS SM. El proceso para

`OPENAI_API_KEY`: la Lambda crea una nueva API key en la API de OpenAI, la almacena en AWS SM, y AWS SM notifica automáticamente al ESO que el secret cambió; ESO actualiza el Kubernetes Secret; los pods leen la nueva key en el siguiente ciclo de sync (máximo 1 minuto). La API key antigua se mantiene válida durante 5 minutos adicionales para asegurar que ningún pod en ejecución la usa antes de invalidarla. Este proceso es completamente automatizado y genera un evento en CloudTrail con el timestamp y el ARN del secret rotado.

Los parámetros no sensibles (`CHUNK_SIZE=512`, `CHUNK_OVERLAP=64`, `TOP_K=5`, `MAX_ITERATIONS=5`, `RATE_LIMIT_PER_MINUTE=10`, `FAITHFULNESS_THRESHOLD=0.85`) se gestionan con ConfigMaps de Kubernetes, con un ConfigMap por servicio y por entorno. Los valores en los ConfigMaps de prod se diferencian de los de dev donde es apropiado: `RATE_LIMIT_PER_MINUTE` es 10 en prod y 100 en dev (para facilitar el desarrollo sin el rate limiting siendo un obstáculo). Los ConfigMaps se versionan en el repositorio como parte de los manifests de Kubernetes en `infra/k8s/`, por lo que cada cambio de parámetro tiene un commit asociado con autor, fecha y mensaje que explica el cambio.

La validación de configuración al inicio de la aplicación usa Pydantic Settings: la clase `Settings` define todos los parámetros esperados con tipos, valores por defecto y validadores. Si la aplicación arranca sin `OPENAI_API_KEY` (por un error en la configuración del Secret), `Settings` lanza una `ValidationError` inmediatamente — el pod falla el startup check y Kubernetes lo reinicia, y el problema queda registrado en los logs del pod. Este patrón de "falla rápido ante configuración incompleta" es más seguro que arrancar con configuración parcial y fallar más tarde cuando se intenta la primera llamada a la API.

Gestión de configuraciones por capa

- **Secrets en AWS Secrets Manager:** `OPENAI_API_KEY`, `QDRANT_API_KEY`, `DATABASE_URL`, `JWT_PRIVATE_KEY` — sincronizados a Kubernetes Secrets via External Secrets Operator con sincronización continua.
- **Rotación automática:** AWS SM Secret Rotation con Lambda function para APIs de OpenAI y Cohere; rotación cada 90 días; proceso sin downtime con ventana de validez dual de 5 minutos.
- **ConfigMaps de Kubernetes:** parámetros no sensibles versionados en el repositorio en `infra/k8s/configmaps/`; diferenciados por entorno; cambios con commit message descriptivo para audit trail.
- **Pydantic Settings:** validación de configuración completa al startup — falla fast con `ValidationError` si cualquier variable requerida está ausente o tiene formato incorrecto.

- **.env.example** : archivo con todas las variables necesarias documentadas con descripciones, valores de ejemplo seguros para desarrollo, y notas sobre cómo obtener las credenciales reales.
- **CloudTrail**: cada cambio en AWS Secrets Manager emite un evento a CloudTrail con timestamp, ARN del secret y identidad del principal que realizó el cambio — audit trail completo de cambios de configuración.
- **detect-secrets**: hook de pre-commit que escanea cada commit en busca de patrones de secrets en el código — previene que API keys o passwords se commiten accidentalmente.

Nota del Arquitecto: El `.env.example` es el artefacto de incorporación más valioso para nuevos integrantes del equipo. Cuando está bien mantenido — con todas las variables necesarias, una descripción de para qué sirve cada una, y notas sobre cómo obtener las credenciales de desarrollo — un nuevo ingeniero puede hacer `cp .env.example .env` y tener el sistema corriendo localmente en 30 minutos. Cuando está desactualizado — le falta una variable que se añadió recientemente, o tiene una variable que ya no se usa — el nuevo ingeniero pierde horas debuggeando un error críptico de "variable de entorno no definida". Mantener `.env.example` actualizado con cada cambio de configuración debería ser un criterio de aceptación de cualquier PR que modifique la configuración del sistema.

La gestión de configuraciones correcta es el control de seguridad con mayor ROI del sistema: previene el tipo de incidente más frecuente (secret en el código) con un esfuerzo de implementación relativamente bajo. El Capítulo 6 cierra con la síntesis de las prácticas MLOps que convierten el sistema en un producto operacional.

Para recordar: La gestión de secrets es el control de seguridad más frecuentemente comprometido en sistemas de producción — un secret en el código o en un archivo `.env` commitado es una vulnerabilidad crítica que puede explotarse durante años después del commit.

Módulo 12 – Capítulo 06 – Sección 06

Cierre: del sistema funcional al sistema operacional

El Capítulo 6 construyó el puente entre el sistema que funciona en un entorno de desarrollo y el sistema que opera en producción de forma reproducible, segura y con calidad garantizada en cada deploy. Los cinco componentes de este capítulo — containerización, CI/CD con gate de evaluación, infraestructura como código, estrategia canary y gestión de configuraciones — son colectivamente lo que distingue a un AI Engineer de un prototipador de IA. El prototipador puede hacer funcionar el LLM en un notebook; el AI Engineer puede garantizar que ese sistema sigue funcionando con calidad medible seis meses después del primer despliegue, con cada cambio validado automáticamente antes de llegar a los usuarios.

La práctica más transformadora del capítulo para equipos que vienen de proyectos de IA sin MLOps es el gate de evaluación en el CI/CD. La primera semana en la que el gate bloquea un deploy porque faithfulness cayó de 0.87 a 0.77 — causado por un cambio aparentemente inocuo en el system prompt del agente — demuestra de forma concreta que los LLMs tienen comportamientos no lineales frente a cambios pequeños en el contexto, y que la evaluación automática es el único mecanismo confiable para detectar esas regresiones antes de que afecten a los usuarios. Sin el gate, ese cambio habría llegado a producción y la degradación de calidad habría sido detectada días después por quejas de usuarios — o nunca, si los usuarios simplemente dejaron de confiar en el sistema sin reportarlo.

La ingeniería de costos como práctica operativa — el monitoreo continuo del costo por petición, la atribución por equipo y las alertas de presupuesto — cierra el loop entre el constraint de diseño (0.02 USD/petición definido en el Capítulo 1) y la realidad operativa. Sin ese monitoreo, el constraint de costo es una intención documentada en el ADR pero sin verificación en producción. Con el monitoreo activo y las alertas configuradas, el constraint es un SLA operativo que el equipo puede comprometerse a cumplir y que el sistema señala automáticamente cuando se aproxima al límite.

El Capítulo 7 comienza con la evaluación end-to-end del sistema desplegado: el framework de métricas en tres capas (RAG, agéntica y de sistema) que convierte el sistema operacional en

un sistema mejorable con evidencia continua.

Lo que el Capítulo 6 implementó

- **Dockerfile multi-stage:** usuario no-root, hash verification de dependencias, health check integrado, configuración uvicorn de producción — imagen reproducible e inmutable para cada deploy.
- **Pipeline CI/CD:** build (multi-stage + Trivy scan) + test (ruff + mypy + pytest 80% + integration + smoke) + evaluate (RAGAS gate sobre 20 muestras del golden dataset) + deploy (kubectl rollout status) — calidad verificada en cada deploy.
- **Infraestructura Terraform:** S3 backend + DynamoDB locking, módulos networking/compute/services, workspaces por entorno, EKS + RDS Multi-AZ + ElastiCache + Qdrant Cloud.
- **Canary deployment con Argo Rollouts:** steps 5% → 25% → 50% → 100% con AnalysisTemplate en Prometheus (error_rate, p95_latency, faithfulness_ragas); rollback automático sub-2-minutos.
- **Ingeniería de costos:** costo por petición desagregado por componente (embedding + reranking + LLM + infraestructura), atribución por equipo, alertas de "costo_hora > 1.5x promedio" y AWS Budgets.
- **Gestión de configuraciones:** AWS Secrets Manager + External Secrets Operator con rotación automática; ConfigMaps versionados en el repositorio; Pydantic Settings con validación al startup; detect-secrets en pre-commit hook.

Nota del Arquitecto: El conjunto de prácticas de este capítulo tiene un costo de implementación real — estimo entre 3 y 5 días de trabajo de un ingeniero senior para configurar el pipeline CI/CD completo con gate de evaluación, el Terraform de infraestructura y la gestión de secrets con ESO. Ese costo se recupera en el primer incidente que el sistema previene: un deploy que degrada faithfulness de 0.87 a 0.74 y es bloqueado por el gate antes de llegar a los usuarios, o una API key que rota automáticamente sin downtime, o una instancia RDS que falla y el Multi-AZ hace failover en 60 segundos sin que el equipo tenga que intervenir. La pregunta no es "¿vale la pena el costo de implementación?" sino "¿cuántos incidentes de producción necesitamos para amortizarlo?". La respuesta habitual es uno.

El deploy automatizado no es una característica opcional — es la condición necesaria para poder iterar sobre el sistema con confianza, sabiendo que cada cambio pasa por las mismas

puertas de calidad y seguridad.

Para recordar: El deploy automatizado no es una característica opcional — es la condición necesaria para poder iterar sobre el sistema con confianza, sabiendo que cada cambio pasa por las mismas puertas de calidad y seguridad.

"Infrastructure as code is the practice of managing and provisioning infrastructure through machine-readable configuration files rather than manual processes." — Kief Morris, Infrastructure as Code